



HIBERNATE

Introduction to Hibernate



- Hibernate is a framework in Java which comes with an abstraction layer and handles the implementations internally.
- The implementations include tasks like writing a query for CRUD operations or establishing a connection with the databases, etc.
- A framework is basically a software that provides abstraction on multiple technologies like JDBC, servlet, etc.
- Hibernate develops persistence logic, which stores and processes the data for longer use.
- It is lightweight and an ORM tool, and most importantly open-source which gives it an edge over other frameworks.

Hibernate 5.0.1 Introduction



- Hibernate framework simplifies the development of java application to interact with the database.
- Hibernate is an open source, lightweight, ORM (Object Relational Mapping) tool.
- An ORM tool simplifies the data creation, data manipulation and data access.
- It is a programming technique that maps the object to the data stored in the database.
- The ORM tool internally uses the JDBC API to interact with the database.

Hibernate Introduction...



- Hibernate is a high-performance Object/Relational persistence and query service.
- It is licensed under the open source GNU Lesser General Public License (LGPL) and is free to download.
- Hibernate takes care of the mapping from Java classes to database tables (and from Java data types to SQL data types),
- It also provides data query and retrieval facilities.

What is JDBC?



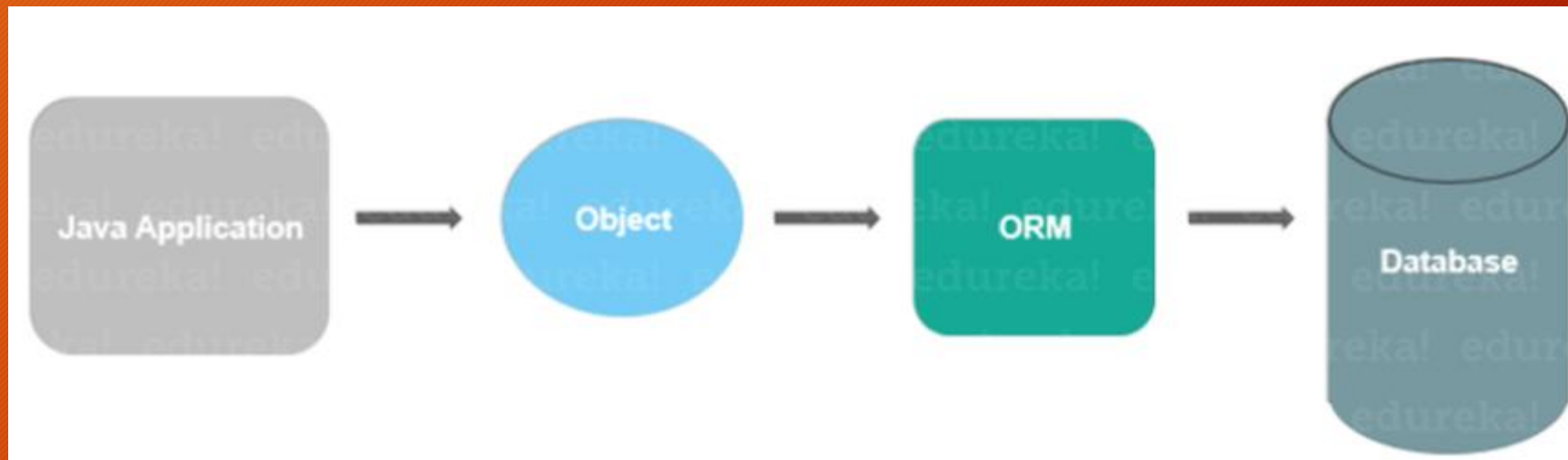
- JDBC stands for **Java Database Connectivity** and provides a set of Java API for accessing the relational databases from Java program.
- These Java APIs enables Java programs to execute SQL statements and interact with any SQL compliant database.

Pros of JDBC	Cons of JDBC
• Clean and simple SQL processing • Good performance with large data • Very good for small applications • Simple syntax so easy to learn	• Complex if it is used in large projects • Large programming overhead • No encapsulation • Hard to implement MVC concept • Query is DBMS specific

Why Object Relational Mapping (ORM) ?



- In an object-oriented systems, there's a mismatch between the object model and the relational database.
- RDBMSs represent data in a tabular format .
- Object-oriented languages, such as Java or C# represent it as an interconnected graph of objects.



Why ORM ? Consider a Java Class with proper constructors and associated public function



```
public class Employee {  
    private int id;  
    private String first_name;  
    private String last_name;  
    private int salary;  
  
    public Employee() {}  
    public Employee(String fname, String lname, int salary)  
    {  
        this.first_name = fname;  
        this.last_name = lname;  
        this.salary = salary;  
    }  
}
```

```
    public int getId() {  
        return id;  
    }  
    public String  
    getFirstName() {  
        return first_name;  
    }  
    public String getLastName()  
    {  
        return last_name;  
    }  
    public int getSalary() {  
        return salary;  
    }  
}
```

Why ORM ?



- Consider above objects need to be stored and retrieved into the following RDBMS table:

```
create table EMPLOYEE (  
  id INT NOT NULL auto_increment,  
  first_name VARCHAR(20) default NULL,  
  last_name VARCHAR(20) default NULL,  
  salary INT default NULL,  
  PRIMARY KEY (id)  
);
```

Problem - What if we need to modify the design of our database after having developed few pages or our application?

ORM - is the solution to handle below impedance mismatches.



- Loading and storing objects in a relational database exposes us to the following five mismatch problems.

Mismatch	Description
Granularity	Sometimes you will have an object model which has more classes than the number of corresponding tables in the database.
Inheritance	RDBMSs do not define anything similar to Inheritance which is a natural paradigm in object-oriented programming languages.
Identity	A RDBMS defines exactly one notion of 'sameness': the primary key. Java, however, defines both object identity (<code>a==b</code>) and object equality (<code>a.equals(b)</code>).
Associations	Object-oriented languages represent associations using object references where as an RDBMS represents an association as a foreign key column.
Navigation	The ways you access objects in Java and in a RDBMS are fundamentally different.

What is ORM?



- ORM stands for Object-Relational Mapping .
- (ORM) is a programming technique for converting data between relational databases and object oriented programming languages such as Java, C# etc.

An ORM system has following advantages over plain JDBC

- Lets business code access objects rather than DB tables.
- Hides details of SQL queries from OO logic.
- Based on JDBC 'under the hood'
- No need to deal with the database implementation.
- Entities based on business concepts rather than database structure.
- Transaction management and automatic key generation.
- Fast development of application.

ORM Solution -4 Entities



- An API to perform basic CRUD operations on objects of persistent classes.
- A language or API to specify queries that refer to classes and properties of classes.
- A configurable facility for specifying mapping metadata.
- A technique to interact with transactional objects to perform dirty checking, lazy association fetching, and other optimization functions.

Java ORM Frameworks



- There are several persistent frameworks and ORM options in Java.
- A persistent framework is an ORM service that stores and retrieves objects into a relational database.
 - **Enterprise JavaBeans Entity Beans**
 - **Java Data Objects**
 - **Castor**
 - **TopLink**
 - **Spring DAO**
 - **Hibernate**
 - **And many more**

Hibernate - Overview



- Hibernate is an Object-Relational Mapping(ORM) solution for JAVA.
- It raised as an open source persistent framework created by **Gavin King** in 2001.
- It is a powerful, high performance **Object-Relational Persistence** and **Query service** for any Java Application.
- Hibernate maps Java classes to database tables and from Java data types to SQL data types.
- Hence relieve the developer from 95% of common data persistence related programming tasks.

Hibernate - Overview...



- Hibernate sits between traditional Java objects and database server.
- It handles all the work in persisting those objects based on the appropriate O/R mechanisms and patterns.



Hibernate Advantages



- Hibernate takes care of mapping Java classes to database tables using XML files and without writing any line of code.
- Provides simple APIs for storing and retrieving Java objects directly to and from the database.
- If there is change in Database or in any table then the only need to change XML file properties.
- Abstract away the unfamiliar SQL types and provide us to work around familiar Java Objects.

Hibernate Advantages



- Hibernate does not require an application server to operate.
- Manipulates Complex associations of objects of your database.
- Minimize database access with smart fetching strategies.
- Provides Simple querying of data.

Hibernate Advantages



- **Opensource and Lightweight:** Hibernate framework is opensource under the LGPL license and lightweight.
- **Fast performance:** The performance of hibernate framework is fast because cache is internally used in hibernate framework.
 - There are two types of cache in hibernate framework first level cache and second level cache. First level cache is enabled by default.
- **Database Independent query:** HQL (Hibernate Query Language) is the object-oriented version of SQL.
 - It generates the database independent queries.
 - So you don't need to write database specific queries.
 - Before Hibernate, If database is changed for the project, we need to change the SQL query as well that leads to the maintenance problem.

Hibernate Advantages



- **Automatic table creation:** Hibernate framework provides the facility to create the tables of the database automatically. So there is no need to create tables in the database manually.
- **Simplifies complex join:** To fetch data from multiple tables is easy in hibernate framework.
- **Provides query statistics and database status:** Hibernate supports Query cache and provide statistics about query and database status.

Supported Databases



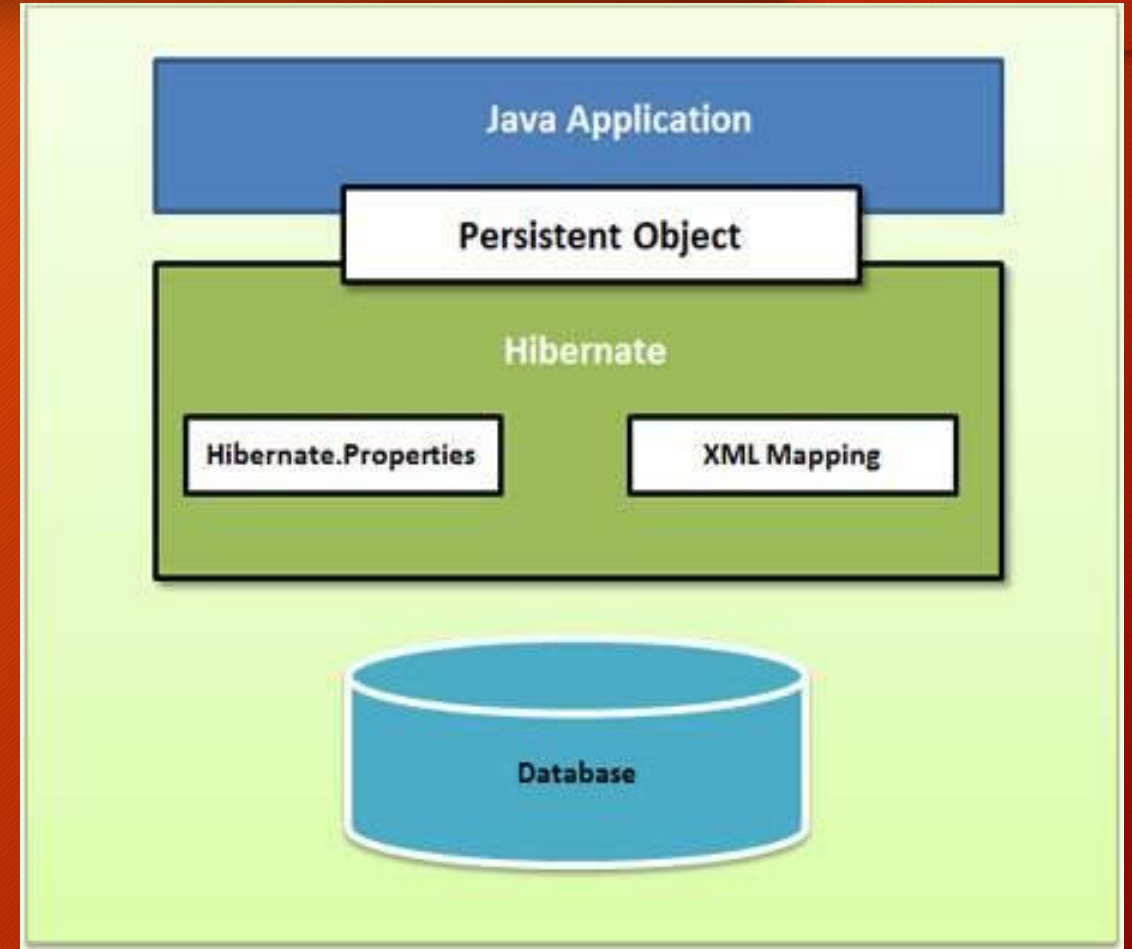
Hibernate supports almost all the major RDBMS.

- HSQL Database Engine
- DB2/NT
- MySQL
- PostgreSQL
- FrontBase
- Oracle
- Microsoft SQL Server Database
- Sybase SQL Server
- Informix Dynamic Server

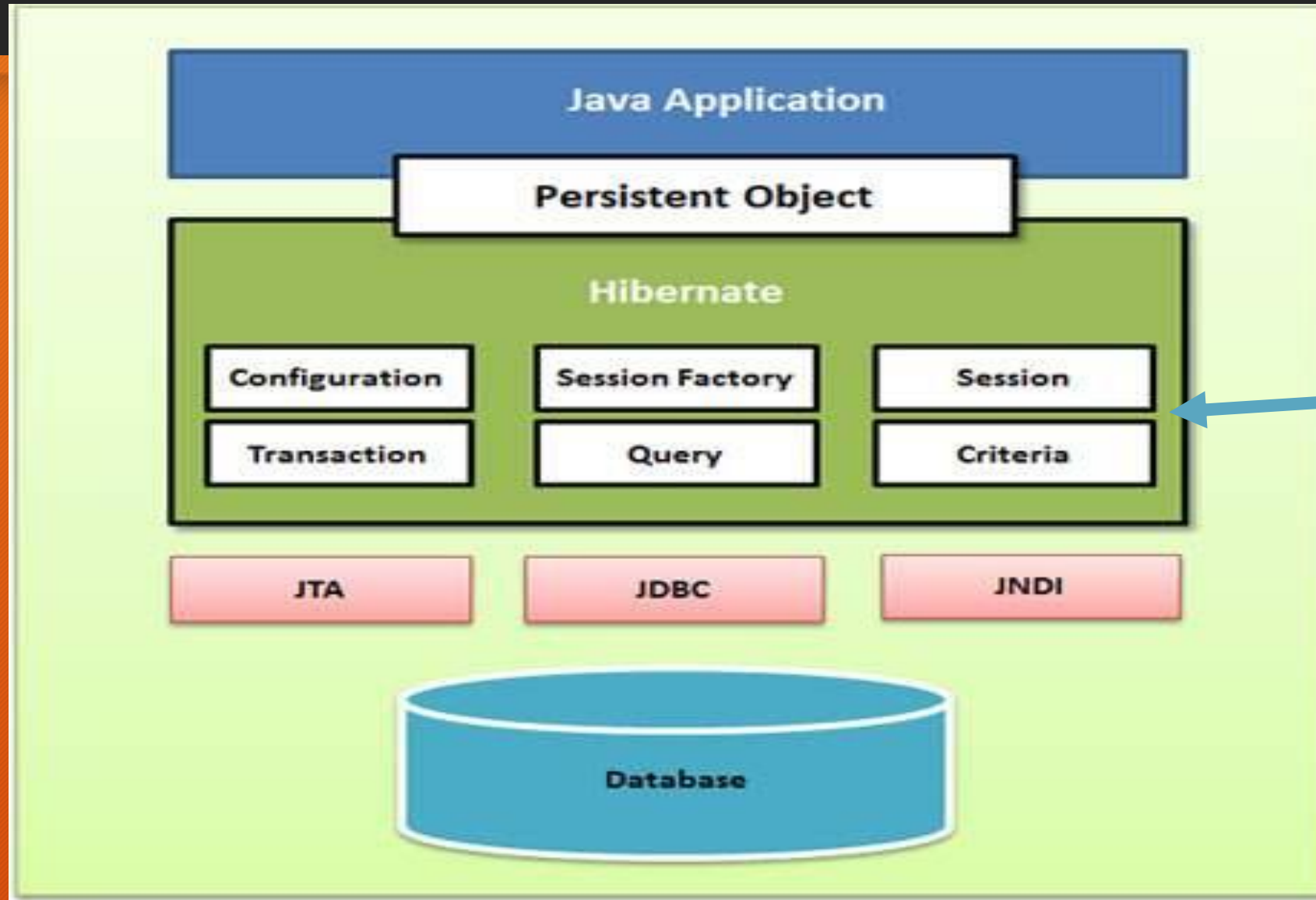
Hibernate - Architecture



- The Hibernate architecture is layered to keep you isolated from having to know the underlying APIs.
- Hibernate makes use of the database and configuration data to provide persistence services (and persistent objects) to the application.



Detailed view of the Hibernate Application Architecture



Core
Classes

Hibernate Architecture - API's



- Hibernate uses various existing Java APIs, like **JDBC**, Java Transaction API(**JTA**), and Java Naming and Directory Interface (**JNDI**).
- JDBC provides a rudimentary level of abstraction of functionality common to relational databases, allowing almost any database with a JDBC driver to be supported by Hibernate.
- JNDI and JTA allow Hibernate to be integrated with J2EE application servers.

Configuration Object



- The Configuration object is the first Hibernate object you create in any Hibernate application.
- It is usually created only once during application initialization.
- It represents a configuration or properties file required by the Hibernate.
- The Configuration object provides two key components:
 - **Database Connection:** This is handled through one or more configuration files supported by Hibernate.
These files are `hibernate.properties` and `hibernate.cfg.xml`.
 - **Class Mapping Setup**
- This component creates the connection between the Java classes and database tables.

SessionFactory Object



- Configuration object is used to create a **SessionFactory** object.
- It in turn configures Hibernate for the application using the supplied configuration file and allows for a Session object to be instantiated.
- The SessionFactory is a thread safe object and used by all the threads of an application.
- The SessionFactory is heavyweight object so usually it is created during application start up and kept for later use.
- You would need one SessionFactory object per database using a separate configuration file.
- So if you are using multiple databases then you would have to create multiple SessionFactory objects.

Session Object



- A Session is used to get a **physical connection** with a database.
- The Session object is lightweight and designed to be instantiated each time an interaction is needed with the database.
- **Persistent objects** are saved and retrieved through a Session object.
- The session objects should not be kept open for a long time because they are not usually thread safe and they should be created and destroyed them as needed.

Transaction Object



A Transaction represents a unit of work with the database and most of the RDBMS supports transaction functionality.

Transactions in Hibernate are handled by an underlying transaction manager and transaction (from JDBC or JTA).

This is an optional object.

Hibernate applications may choose not to use this interface, instead managing transactions in their own application code.

Query Object



Query objects use SQL or Hibernate Query Language (HQL) string to retrieve data from the database and create objects.

A Query instance is used to bind query parameters, limit the number of results returned by the query, and finally to execute the query.

Criteria Object



- Criteria object are used to create and execute object oriented criteria queries to retrieve objects.

Hibernate - Environment



- JDK latest Version
- Eclipse IDE
- Download the latest version of Hibernate from <http://www.hibernate.org/downloads>
- Download Hibernate Prerequisites
 - **dom4j** - XML parsing www.dom4j.org/
 - **Xalan** - XSLT Processor <http://xml.apache.org/xalan-j/>
 - **Xerces** - The Xerces Java Parser <http://xml.apache.org/xerces-j/>
 - **cglib** - Appropriate changes to Java classes at runtime <http://cglib.sourceforge.net/>
 - **log4j** - Logging Framework <http://logging.apache.org/log4j>
 - **Commons** - Logging, Email etc. <http://jakarta.apache.org/commons>
 - **SLF4J** - Logging Facade for Java <http://www.slf4j.org>

Hibernate - Environment with Maven



- JDK latest Version
- Eclipse IDE with Hibernate Tools - **Jboss Tools**
- In pom.xml add Hibernate and Database Dependencies from mvnrepository.com

```
<dependency>
<groupId>org.hibernate</groupId>
<artifactId>hibernate-core</artifactId>
<version>6.4.1.Final</version>
</dependency>

<dependency>
<groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
<version>8.0.33</version>
</dependency>
```

Hibernate - Configuration



- Hibernate requires to know in advance where to find the mapping information that defines how your Java classes relate to the database tables.
- Hibernate also requires a set of configuration settings related to database and other related parameters.
- All such information is usually supplied as a standard Java properties file called **hibernate.properties**, or as an XML file named **hibernate.cfg.xml**.

hibernate.cfg.xml



- We will consider XML formatted file **hibernate.cfg.xml** to specify required Hibernate properties.
- Most of the properties take their default values and it is not required to specify them in the property file unless it is really required.
- This file is kept in the root directory of your application's classpath.

Hibernate Properties



Properties	Description
<code>hibernate.dialect</code>	This property makes Hibernate generate the appropriate SQL for the chosen database.
<code>hibernate.connection.driver_class</code>	The JDBC driver class.
<code>hibernate.connection.url</code>	The JDBC URL to the database instance.
<code>hibernate.connection.username</code>	The database username.
<code>hibernate.connection.password</code>	The database password.
<code>hibernate.connection.pool_size</code>	Limits the number of connections waiting in the Hibernate database connection pool.
<code>hibernate.connection.autocommit</code>	Allows autocommit mode to be used for the JDBC connection.

Hibernate with MySQL Database



```
<hibernate-configuration>
  <session-factory>
    <property name="hbm2ddl.auto">create</property>
    <property name="dialect">org.hibernate.dialect.MySQL5Dialect </property>
    <property name="connection.url"> jdbc:mysql://localhost:3306/mysqljdbc </property>
    <property name="connection.username">root</property>
    <property name="connection.password">redhat</property>
    <property name="connection.driver_class"> com.mysql.jdbc.Driver</property>
    <mapping resource="user.hbm.xml"/>
  </session-factory>
</hibernate-configuration>
```

Databases dialect property



Database	Dialect Property
DB2	org.hibernate.dialect.DB2Dialect
HSQLDB	org.hibernate.dialect.HSQLDialect
HypersonicSQL	org.hibernate.dialect.HSQLDialect
Informix	org.hibernate.dialect.InformixDialect
Ingres	org.hibernate.dialect.IngresDialect
Interbase	org.hibernate.dialect.InterbaseDialect
Microsoft SQL Server 2000	org.hibernate.dialect.SQLServerDialect
Microsoft SQL Server 2005	org.hibernate.dialect.SQLServer2005Dialect
Microsoft SQL Server 2008	org.hibernate.dialect.SQLServer2008Dialect
MySQL	org.hibernate.dialect.MySQLDialect
Oracle (any version)	org.hibernate.dialect.OracleDialect
Oracle 11g	org.hibernate.dialect.Oracle10gDialect
Oracle 10g	org.hibernate.dialect.Oracle10gDialect
Oracle 9i	org.hibernate.dialect.Oracle9iDialect
PostgreSQL	org.hibernate.dialect.PostgreSQLDialect
Progress	org.hibernate.dialect.ProgressDialect
SAP DB	org.hibernate.dialect.SAPDBDialect
Sybase	org.hibernate.dialect.SybaseDialect
Sybase Anywhere	org.hibernate.dialect.SybaseAnywhereDialect

Hibernate - Sessions



- A Session is used to get a physical connection with a database.
- The Session object is lightweight and designed to be instantiated each time an interaction is needed with the database.
- Persistent objects are saved and retrieved through a Session object.
- The session objects should not be kept open for a long time because they are not usually thread safe and they should be created and destroyed them as needed.
- The main function of the Session is to offer create, read and delete operations for instances of mapped entity classes.

Session object - Instance



- **transient:** A new instance of a persistent class which is not associated with a Session and has no representation in the database and no identifier value is considered transient by Hibernate.
- **persistent:** You can make a transient instance persistent by associating it with a Session. A persistent instance has a representation in the database, an identifier value and is associated with a Session.
- **detached:** Once we close the Hibernate Session, the persistent instance will become a detached instance.

Session instance



- A Session instance is serializable if its persistent classes are serializable.
- A typical transaction should use the following idiom

```
Session session = factory.openSession();
Transaction tx = null;
try {
    tx = session.beginTransaction();
    // do some work
    ...
    tx.commit();
}
catch (Exception e) {
    if (tx!=null) tx.rollback();
    e.printStackTrace();
}finally {
    session.close();
}
```


Hibernate - Persistent Class



- The entire concept of Hibernate is to take the values from Java class attributes and persist them to a database table.
- A mapping document helps Hibernate in determining how to pull the values from the classes and map them with table and associated fields.
- Java classes whose objects or instances will be stored in database tables are called **persistent classes** in Hibernate.
- Hibernate works best if these classes follow some simple rules, also known as the Plain Old Java Object (POJO) programming model.

Rules of persistent classes



- All Java classes that will be persisted need a default constructor.
- All classes should contain an ID in order to allow easy identification of your objects within Hibernate and the database.
- This property maps to the primary key column of a database table.
- All attributes that will be persisted should be declared private and have `getXXX` and `setXXX` methods defined in the JavaBean style.
- A central feature of Hibernate, proxies, depends upon the persistent class being either non-final, or the implementation of an interface that declares all public methods.
- All classes that do not extend or implement some specialized classes and interfaces required by the EJB framework.

A simple POJO example



```
public class Employee {  
    private int id;  
    private String firstName;  
    private String lastName;  
    private int salary;  
    public Employee() {}  
    public Employee(String fname, String lname, int salary) {  
        this.firstName = fname;  
        this.lastName = lname;  
        this.salary = salary; }  
    public int getId() {  
        return id; }  
    public void setId( int id ) {  
        this.id = id; }  
    public String getFirstName() {  
        return firstName; }  
}
```

```
    public void setFirstName( String first_name ) {  
        this.firstName = first_name;  
    }  
    public String getLastName() {  
        return lastName;  
    }  
    public void setLastName( String last_name ) {  
        this.lastName = last_name;  
    }  
    public int getSalary() {  
        return salary;  
    }  
    public void setSalary( int salary ) {  
        this.salary = salary;  
    }  
}
```


Hibernate - Mapping Files



- An Object/relational mappings are usually defined in an XML document.
- This mapping file instructs Hibernate how to map the defined class or classes to the database tables.
- Though many Hibernate users choose to write the XML by hand, a number of tools exist to generate the mapping document.
- You should save the mapping document in a file with the format **<classname>.hbm.xml**.

Employee.hbm.xml - Sample Mapping File



```
<hibernate-mapping>
  <class name="Employee" table="EMPLOYEE">
    <meta attribute="class-description">
      This class contains the employee detail.
    </meta>
    <id name="id" type="int" column="id">
      <generator class="native"/>
    </id>
    <property name="firstName" column="first_name" type="string"/>
    <property name="lastName" column="last_name" type="string"/>
    <property name="salary" column="salary" type="int"/>
  </class>
</hibernate-mapping>
```

Employee.hbm.xml - Properties



- The mapping document is an XML document having **<hibernate-mapping>** as the root element which contains all the **<class>** elements.
- The **<class>** elements are used to define specific mappings from a Java classes to the database tables.
 - The Java class name is specified using the name attribute of the class element and the database table name is specified using the table attribute.
- The **<meta>** element is optional element and can be used to create the class description.
- The **<id>** element maps the unique ID attribute in class to the primary key of the database table.
 - The name attribute of the id element refers to the property in the class and the column attribute refers to the column in the database table.
 - The type attribute holds the hibernate mapping type, this mapping types will convert from Java to SQL data type.

Employee.hbm.xml - Properties



- The **<generator>** element within the id element is used to automatically generate the primary key values.
 - Set the class attribute of the generator element is set to native to let hibernate pick up either identity, sequence or hilo algorithm to create primary key depending upon the capabilities of the underlying database.
- The **<property>** element is used to map a Java class property to a column in the database table.
 - The name attribute of the element refers to the property in the class and the column attribute refers to the column in the database table.
 - The type attribute holds the hibernate mapping type, this mapping types will convert from Java to SQL data type.

Hibernate - Mapping Types



- When you prepare a Hibernate mapping document, we have seen that you map Java data types into RDBMS data types.
- The types declared and used in the mapping files are not Java data types; they are not SQL database types either.
- These types are called Hibernate mapping types, which can translate from Java to SQL data types and vice versa.

Mapping type	Java type	ANSI SQL Type
integer	int or java.lang.Integer	INTEGER
long	long or java.lang.Long	BIGINT
string	java.lang.String	VARCHAR
class	java.lang.Class	VARCHAR
date	java.util.Date or java.sql.Date	DATE

Generator classes in Hibernate



- The <generator> class is a sub-element of id.
- It is used to generate the unique identifier for the objects of persistent class.
- **Assigned** -It is the default generator strategy if there is no <generator> element , application assigns the id.
- **Increment** -It generates the unique id only if no other process is inserting data into this table. It generates **short**, **int** or **long** type identifier.
- **Native** - It uses identity, sequence depending on the database vendor.
- **Identity** - It is responsibility of database to generate unique identifier.

Hibernate hbm2ddl.auto possible values and their uses



- Automatically validates or exports schema DDL to the database when the SessionFactory is created.
- With create-drop, the database schema will be dropped when the SessionFactory is closed explicitly.
 - **validate**: validate the schema, makes no changes to the database.
 - **update**: creates schema, if it doesn't exist and update the schema and data.
 - **create**: creates the schema, destroying previous data.
 - **create-drop**: drop the schema when the SessionFactory is closed explicitly, typically when the application is stopped.

Create Hibernate Application in Eclipse IDE



- Create the java/Maven project
- Add jar files for hibernate/ add dependency in pom.xml
- Create the Persistent class
- Create the mapping file for Persistent class
- Create the Configuration file
- Create the class that retrieves or stores the persistent object
 - **import org.hibernate.*;**
- Run the application

Mapping in Hibernate



- Mapping in Hibernate refers to the process of defining the relationship between a Java class and a database table.
- It is done using annotations, XML files, or both.
- Hibernate mapping allows you to specify how a Java class attribute maps to a database column, and how relationships between classes map to relationships between tables.
- For example, you can use the `@Entity` annotation to define a Java class as an entity, and the `@Column` annotation to specify the mapping of a class attribute to a database column.
- This enables you to manage complex data relationships in your database in a flexible and efficient way.

Hibernate - O/R Mappings



- **Most important mapping :**
 - **Collections Mappings**
 - **Association Mappings**
 - **Component Mappings**
 - **Inheritance Mappings**

Collections Mappings



- If an entity or class has collection of values for a particular variable, then we can map those values using any one of the collection interfaces available in java.
- Hibernate can persist instances of `java.util.Map`, `java.util.Set`, `java.util.SortedMap`, `java.util.SortedSet`, `java.util.List`, and any array of persistent entities or values.

```
public class Employee {  
  
    private int employeeId;  
    private String employeeName;  
    private Set<String> phoneNumbers; }  

```

Collections Mappings using Annotations



- Hibernate provides several annotations for mapping collections within an entity, depending on whether the collection contains basic types, embeddable types, or other entities.
- 1. Collections of Basic or Embeddable Types:

```
@Entity
public class User {
    @Id
    private Long id;
    private String name;

    @ElementCollection
    @CollectionTable(name = "user_emails", joinColumns = @JoinColumn(name = "user_id"))
    @Column(name = "email_address")
    private Set<String> emails;    // Getters and Setters}
```


Collections Mappings using Annotations



- **@ElementCollection:** Marks the field as a collection of basic or embeddable types.
- **@CollectionTable:** Specifies the name of the separate table for the collection and the foreign key column (joinColumns) linking it to the main entity's table.
- **@Column:** Specifies the column name for the elements within the collection table.
- For List types, **@OrderColumn** can be used to store the index of each element.
- For Map types, **@MapKeyColumn** can be used to specify the column for the map's keys.

Component Mappings



- It is very much possible that an Entity class can have a reference to another class as a member variable.
- If the referred class does not have its own life cycle and completely depends on the life cycle of the owning entity class, then the referred class hence therefore is called as the Component class.
- The mapping of Collection of Components is also possible in a similar way just as the mapping of regular Collections with minor configuration differences.
- **Component Mappings** - Mapping for a class having a reference to another class as a member variable.

Component Mapping - Example



- Java supports composition in the form of has-a relationship.
- Composition is stronger association where the contained object has no existence of its own.
- For example Person has an address, Person has a job . In these examples job and address have no existence without Person.
- We may want to store contained objects like Job, Address etc in the same entity table (person) as a value type and Hibernate does support this feature using **<component>** tag.
- Most important thing is - contained object is not stored in a separate table instead it will be stored as a value type in table corresponding to contained object and we can have nested component mapping.

Component Mapping using Annotations



- Component mapping in Hibernate using annotations allows you to embed a value object (a class without its own identity, whose lifecycle is dependent on its owning entity) directly into the owning entity's table.
- This is achieved using the `@Embeddable` and `@Embedded` annotations from JPA.

Create the Embeddable Class
(Component):

```
@Embeddable
public class Address {
    private String street;
    private String city;
    private String zipCode;

    // Getters and setters
}
```

Embed the Component in the Owning Entity



- In your main entity class (e.g., Employee, Customer), declare a field of the embeddable class type.
- Annotate this field with `@Embedded`. This tells Hibernate to embed the properties of the Address class directly into the Employee table.

```
@Entity
@Table(name = "EMPLOYEE")
public class Employee {
    @Id
    @GeneratedValue(strategy =
        GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;

    @Embedded
    private Address address; // Embedding the
                             Address component

    // Getters and setters
}
```

Component Mapping Annotations



- **@Embeddable:** This annotation marks a class as suitable for being embedded within another entity.
- It essentially means this class doesn't have its own primary key and its data will be stored as part of the owning entity's table.
- **@Embedded:** This annotation is placed on a field within an entity to indicate that the field's type is an @Embeddable class, and its properties should be mapped directly to columns in the owning entity's table.

Inheritance Mapping In Hibernate



- Java is object oriented language and inheritance is one of main functionalities of java.
- Relation model can implement "is a" and "has a" relationship but hibernate provides us way to implement class hierarchy in a different ways.
- In this mapping if we have base and derived classes, now if we save derived(sub) class object, base class object will also be stored into the database.

```
<subclass name="Candidate" extends="Person" discriminator-value="E">  
<property name="departmentName" column="department_name" />  
<property name="joiningDate" type="date" column="joining_date" />  
</subclass>
```

Hibernate - @Inheritance Annotation



- The **@Inheritance** annotation in JPA is used to specify the inheritance relation between two entities.
- It is used to define how the data of the entities in the hierarchy should be stored in the database.
- The **@Inheritance** annotation provides us with benefits to reduce code complexity by creating a base class and inheriting other classes from the base class.
- The **@Inheritance** annotation is applied to the root entity class to define the inheritance strategy.

Hibernate - @Inheritance Annotation



There are different types of strategies available for Inheritance annotation which are as follows:

- Single Table Inheritance Strategy.
- Joined Inheritance Strategy.
- Table Per Class Inheritance Strategy.

Single Table Inheritance Strategy



- The Single Table Inheritance strategy states that all the entities in this hierarchy are mapped to a single table in the database.
- Separate columns are used to differentiate between multiple entities.
- This strategy is suitable when entities in the hierarchy share most of their attributes and have a small number of different fields.

```
// on the below line creating an entity for Student
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "entity_type",
                    discriminatorType
                    = DiscriminatorType.STRING)
public class Student {
```

Joined Inheritance Strategy



- The Joined Inheritance Strategy generates a separate table for each entity class.
- The attribute for each table is joined with the primary key. It removes the possibility of duplicity.
- Each table that is being created contains specific columns and a join operation is used to retrieve data from multiple tables when querying the entire hierarchy.
- We can use the strategy when the entities we are creating have differences in their properties and relationships.

```
@Entity
// on the below line we are specifying the inheritance
// strategy as joined.
@Inheritance(strategy = InheritanceType.JOINED)
public class Student {
    // on the below line creating a field for id which we
    // are anoting it with @Id.
    @Id
    private int id;
    // on the below line creating a field for student name.
    private String studentName;
}
```

Association Mappings



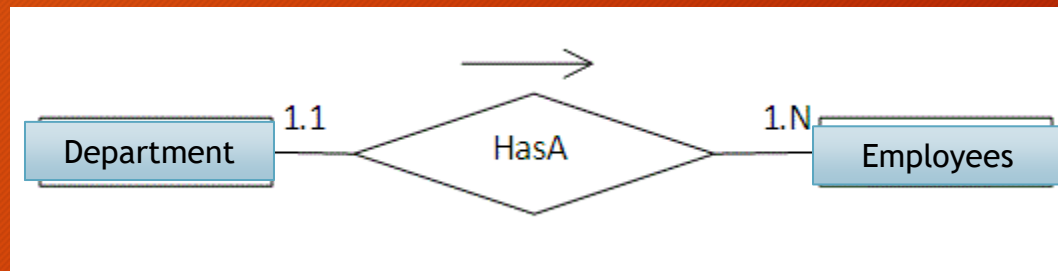
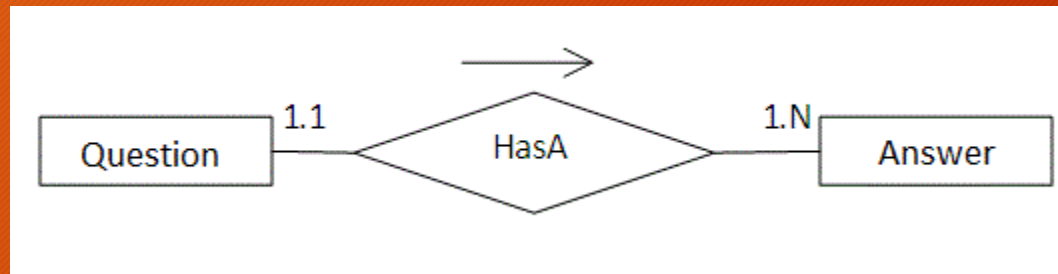
- The mapping of associations between entity classes and the relationships between tables is the soul of ORM.
- Following are the four ways in which the cardinality of the relationship between the objects can be expressed.
- An association mapping can be unidirectional as well as bidirectional.

Mapping type	Description
Many-to-One	Mapping many-to-one relationship using Hibernate
One-to-One	Mapping one-to-one relationship using Hibernate
One-to-Many	Mapping one-to-many relationship using Hibernate
Many-to-Many	Mapping many-to-many relationship using Hibernate

Hibernate One to Many



- If the persistent class has list object that contains the entity reference, we need to use one-to-many association to map the list element.
- Here, we are using the scenario of Forum where one question has multiple answers.



Annotations in One-Many



- @OneToMany(cascade = CascadeType.ALL
- @JoinColumn(name="qid")
- @OrderColumn(name="type")

Hibernate One to One



- To perform one to one mapping by one-to-one element using annotation.
- In such case, no foreign key is created in the primary table.
- Example, one employee can have one address and one address belongs to one employee only.
- One Employee is assigned one parking lot.
- `@OneToOne(targetEntity=Address.class,cascade=CascadeType.ALL)`
- `@OneToOne(targetEntity=Employee.class)`

Hibernate - Query Language



- **Hibernate Query Language (HQL) is an object-oriented query language, similar to SQL.**
- **Instead of operating on tables and columns, HQL works with persistent objects and their properties.**
- **HQL queries are translated by Hibernate into conventional SQL queries which in turns perform action on database.**
- **We can use SQL statements directly with Hibernate using Native SQL.**
- **Use HQL whenever possible to avoid database portability hassles, and to take advantage of Hibernate's SQL generation and caching strategies.**

Advantage of HQL



- Database independent
- Supports polymorphic queries
- Easy to learn for Java Programmer

Query Object



Query objects use SQL or Hibernate Query Language (HQL) string to retrieve data from the database and create objects.

A Query instance is used to bind query parameters, limit the number of results returned by the query, and finally to execute the query.

Query Interface - Methods



- The object of Query can be obtained by calling the **createQuery()** method Session interface.
- **public int executeUpdate()** is used to execute the update or delete query.
- **public List list()** returns the result of the relation as a list.
- **public Query setFirstResult(int rowno)** specifies the row number from where record will be retrieved.
- **public Query setMaxResult(int rowno)** specifies the no. of records to be retrieved from the relation (table).
- **public Query setParameter(int position, Object value)** it sets the value to the JDBC style query parameter.
- **public Query setParameter(String name, Object value)** it sets the value to a named query parameter.

HQL- FROM Clause



- Use FROM clause if you want to load a complete persistent objects into memory.

```
String hql = "FROM Employee";  
Query query =  
session.createQuery(hql,Employee.class);  
List results = query.list();
```

HQL- AS Clause



- The AS clause can be used to assign aliases to the classes in your HQL queries, specially when you have long queries.

```
String hql = "FROM Employee AS E";  
Query query = session.createQuery(hql);  
List results = query.list();
```


HQL- SELECT Clause



- The SELECT clause provides more control over the result set than the from clause.
- If you want to obtain few properties of objects instead of the complete object, use the SELECT clause.

```
String hql = "SELECT E.firstName FROM Employee E";  
Query query = session.createQuery(hql);  
List results = query.list();
```

Note : Employee.firstName is a property of Employee object rather than a field of the EMPLOYEE table.

HQL-WHERE Clause



- If you want to narrow the specific objects that are returned from storage, you use the WHERE clause.

```
String hql = "FROM Employee E WHERE E.id = 10";  
Query query = session.createQuery(hql);  
List results = query.list();
```

HQL-ORDER BY Clause



- To sort your HQL query's results, you will need to use the ORDER BY clause.
- You can order the results by any property on the objects in the result set either ascending (ASC) or descending (DESC).

```
String hql = "FROM Employee E WHERE E.id > 10 ORDER BY E.salary DESC";  
Query query = session.createQuery(hql);  
List results = query.list();
```


HQL-GROUP BY Clause



- This clause lets Hibernate pull information from the database and group it based on a value of an attribute .
- typically, use the result to include an aggregate value.

```
String hql = "SELECT SUM(E.salary), E.firtName FROM Employee E " + "GROUP BY  
E.firstName";  
Query query = session.createQuery(hql);  
List results = query.list();
```

HQL- Using Named Parameters



- Hibernate supports named parameters in its HQL queries.
- This makes writing HQL queries that accept input from the user easy and you do not have to defend against SQL injection attacks.

```
String hql = "FROM Employee E WHERE E.id = :employee_id";  
Query query = session.createQuery(hql);  
query.setParameter("employee_id",10);  
List results = query.list();
```

HQL - UPDATE Clause



- The UPDATE clause can be used to update one or more properties of an one or more objects.
- Bulk updates are new to HQL with Hibernate 3, and deletes work differently in Hibernate 3 than they did in Hibernate 2.
- The Query interface now contains a method called `executeUpdate()` for executing HQL UPDATE or DELETE statements.

```
String hql = "UPDATE Employee set salary = :salary " + "WHERE id = :employee_id";
```

```
Query query = session.createQuery(hql);
```

```
query.setParameter("salary", 1000);
```

```
query.setParameter("employee_id", 10);
```

```
int result = query.executeUpdate();
```

```
System.out.println("Rows affected: " + result);
```


HQL-DELETE Clause



- The DELETE clause can be used to delete one or more objects.

```
String hql = "DELETE FROM Employee " + "WHERE id = :employee_id";  
Query query = session.createQuery(hql);  
query.setParameter("employee_id", 10);  
int result = query.executeUpdate();  
System.out.println("Rows affected: " + result);
```

HQL-INSERT Clause



- HQL supports INSERT INTO clause only where records can be inserted from one object to another object.

```
String hql = "INSERT INTO Employee(firstName, lastName, salary)" + "SELECT  
firstName, lastName, salary FROM old_employee"; Query query =  
session.createQuery(hql);  
int result = query.executeUpdate();  
System.out.println("Rows affected: " + result);
```

HQL-Aggregate Methods



- HQL supports a range of aggregate methods, similar to SQL.
- The **distinct** keyword only counts the unique values in the row set.

```
String hql = "SELECT count(distinct E.firstName) FROM Employee E";  
Query query = session.createQuery(hql);  
List results = query.list();
```

- **avg(property name)**
- **count(property name or *)**
- **max(property name)**
- **min(property name)**
- **sum(property name)**

Hibernate - Criteria Queries



- Hibernate provides alternate ways of manipulating objects and in turn data available in RDBMS tables.
- One of the methods is **Criteria API**.
- It allows you to build up a criteria query object programmatically.
- You can apply filtration rules and logical conditions .

Criteria Queries



- The Hibernate Session interface provides **createCriteria()** method which can be used to create a Criteria object.
- It returns instances of the persistence object's class when your application executes a criteria query.
- Criteria query is one which will simply return every object that corresponds to the Employee class.

```
Criteria cr = session.createCriteria(Employee.class);  
List results = cr.list();
```

Restrictions with Criteria



- You can use add() method available for Criteria object to add restriction for a criteria query.
- Example to add a restriction to return the records with salary is equal to 2000

```
Criteria cr = session.createCriteria(Employee.class);  
cr.add(Restrictions.eq("salary", 2000));  
  
List results = cr.list();
```

```
cr.add(Restrictions.gt("salary", 2000));
```

```
cr.add(Restrictions.between("salary", 1000, 2000));
```

```
cr.add(Restrictions.like("firstName", "r%"));
```


TypedQuery



- In Hibernate, TypedQuery is a Java Persistence API (JPA) interface used to control the execution of typed queries.
- It is a sub-interface of `javax.persistence.Query`.
- It adds type safety by specifying the expected type of the query results.
- JPA can't deduce what the Query result type will be, and, as a result, we have to cast.
- But, JPA provides a special Query sub-type known as a TypedQuery.
- This is always preferred if we know our Query result type beforehand.
- Additionally, it makes our code much more reliable and easier to test.

Key aspects of TypedQuery



- **Type Safety:**

- The primary advantage of TypedQuery is its type parameter X, which explicitly declares the type of the objects returned by the query.
- This allows for compile-time type checking, reducing the risk of ClassCastException at runtime when retrieving results.

- **Creation:**

- TypedQuery instances are typically created using the EntityManager.createQuery() method, providing a JPQL or HQL query string and the expected result class. For example:

```
TypedQuery<User> query = session.createQuery("SELECT u FROM User u WHERE u.username = :username",  
User.class);
```


Key aspects of TypedQuery



- **Execution:**

- `getResultList()`: Executes a SELECT query and returns the results as a `java.util.List` of the specified type X.
- `getSingleResult()`: Executes a SELECT query that is expected to return exactly one result. If no results or more than one result are found, it throws an exception.

- **Parameter Binding:**

- TypedQuery provides methods like `setParameter()` to bind values to named or positional parameters within the query string, ensuring safe and efficient parameter handling.

- **Pagination and Control:**

- Methods like `setFirstResult()` and `setMaxResults()` enable pagination, allowing retrieval of a specific subset of results.
- Other methods like `setFlushMode()`, `setHint()`, and `setLockMode()` offer fine-grained control over query execution and entity behavior.

Java Annotations



Java Annotation is a tag that represents the metadata.

I.e it is attached with class, interface, methods or fields to indicate some additional information which can be used by java compiler and JVM.

Annotations in Java are a form of metadata that can be added to Java source code.

They provide additional information about program elements like classes, methods, fields, parameters, and local variables.

Annotations begin with the '@' symbol.

Hibernate



JPA

Java Persistence API



HIBERNATE

Introduction to JPA



- A JPA (Java Persistence API) is a specification of Java which is used to access, manage, and persist data between Java object and relational database.
- It is considered as a standard approach for Object Relational Mapping.
- JPA can be seen as a bridge between object-oriented domain models and relational database systems.
- Being a specification, JPA doesn't perform any operation by itself. Thus, it requires implementation.
- So, ORM tools like Hibernate, TopLink, and iBatis implements JPA specifications for data persistence.

JPA

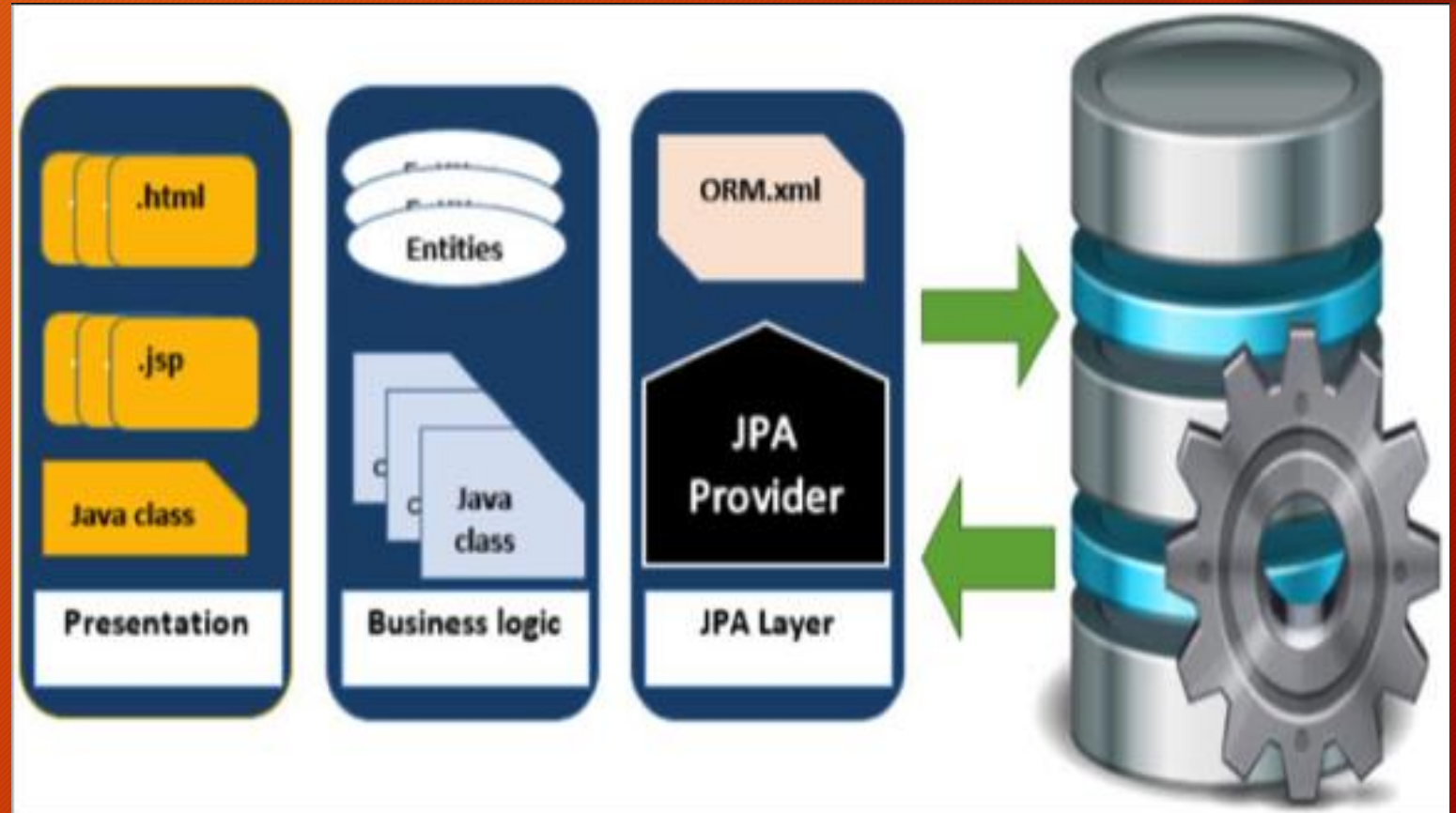


- Java/Jakarta Persistence API is a collection of classes and methods to persistently store the vast amounts of data into a database which is provided by the Oracle Corporation.
- The hibernate application can be created with annotation.
- There are many annotations that can be used to create hibernate application such as **@Entity, @Id, @Table** etc.
- Hibernate Annotations are based on the JPA 2 specification and supports all the features.
- **Hibernate is an implementation of JPA. So, it follows the common standards provided by the JPA.**

Where to use JPA



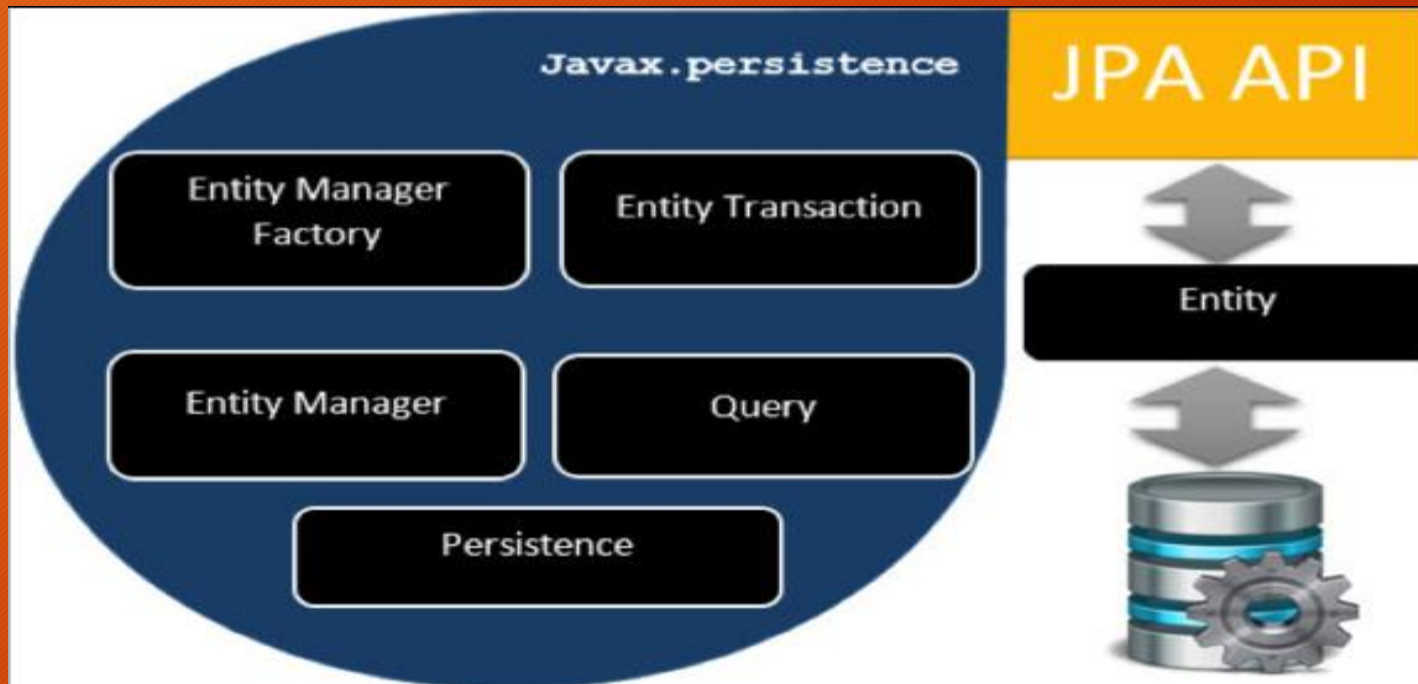
- To reduce the burden of writing codes for relational object management, a programmer follows the 'JPA Provider' framework, which allows easy interaction with database instance.
- Here the required framework is taken over by JPA.



JPA - Architecture



- Java Persistence API is a source to store business entities as relational entities.
- It shows how to define a PLAIN OLD JAVA OBJECT (POJO) as an entity and how to manage entities with relations.

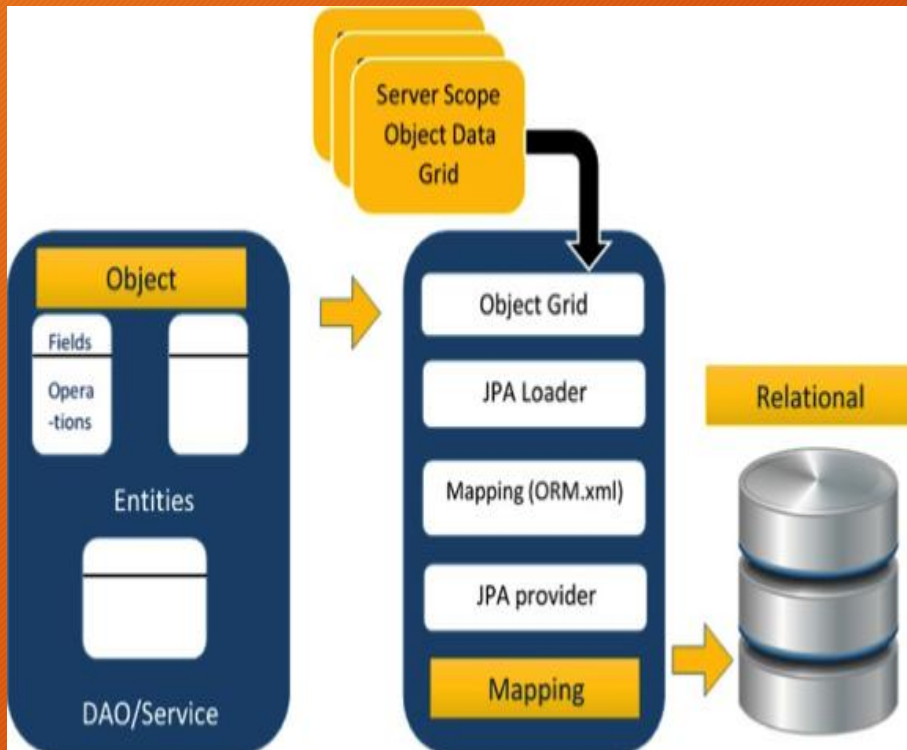


Object Relational Mapping



- Object Relational Mapping (ORM) briefly tells you about what is ORM and how it works.
- ORM is a programming ability to covert data from object type to relational type and vice versa.
- The main feature of ORM is mapping or binding an object to its data in the database.
- While mapping we have to consider the data, type of data and its relations with its self-entity or entity in any other table.

ORM Architecture



- The first phase, named as the **Object data** phase contains POJO classes, service interfaces and classes.
- It is the main business component layer, which has business logic operations and attributes.
- The second phase named as **mapping** or **persistence** phase which contains JPA provider, mapping file (ORM.xml), JPA Loader, and Object Grid.

Mapping.xml



- The mapping.xml file is to instruct the JPA vendor for mapping the Entity classes with database tables.
- **<entity-mappings>** : tag defines the schema definition to allow entity tags into xml file.
- **<entity>** : tag defines the entity class which you want to convert into table in a database. Attribute class defines the POJO entity class name.
- **<id>** : tag defines the primary key of the table.
- The **<generated-value>** tag defines how to assign the primary key value such as Automatic, Manual, or taken from Sequence.

Hibernate Annotations



- We have seen how Hibernate uses XML mapping file for the transformation of data from POJO to database tables and vice versa.
- Hibernate annotations are the newest way to define mappings without the use of XML file.
- You can use annotations in addition to or as a replacement of XML mapping metadata.
- Hibernate Annotations is the powerful way to provide the metadata for the Object and Relational Table mapping.
- All the metadata is clubbed into the POJO java file along with the code.
- If you want greater flexibility, then you should go with XML-based mappings.

Environment Setup for Hibernate Annotation



- JDK 1.5 & above
- hibernate-annotations.jar,
- hibernate-comons-annotations.jar
- ejb3-persistence.jar

Hibernate using Annotation



- The hibernate application can be created with annotation.
- There are many annotations that can be used to create hibernate application such as @Entity, @Id, @Table etc.
- Hibernate Annotations are based on the JPA 2 specification and supports all the features.
- All the JPA annotations are defined in the **javax.persistence package**.
- Hibernate EntityManager implements the interfaces and life cycle defined by the JPA specification.
- The core advantage of using hibernate annotation is that you don't need to create mapping (hbm) file.
- Here, hibernate annotations are used to provide the meta data.

Annotations in Hibernate



- **@Entity Annotation**- Marks a class as an entity bean, so it must have a no-argument constructor that is visible with at least protected scope.
- **@Table Annotation** - It allows you to specify the details of the table that will be used to persist the entity in the database.
- **@Id** - Each entity bean will have a primary key, which you annotate on the class with the @Id annotation. The primary key can be a single field or a combination of multiple fields depending on your table structure.
- **@GeneratedValue** - which takes two parameters **strategy** and **generator type**.
- **@Column Annotation** - is used to specify the details of the column to which a field or property will be mapped.

Persistence class



```
import javax.persistence.Entity;  
import javax.persistence.Id;  
import javax.persistence.Table;
```

```
@Entity
```

```
@Table(name= "emp500")
```

```
public class Employee {
```

```
    @Id
```

```
    private int id;
```

```
    private String firstName,lastName;
```


Hibernate Sequence Generator



@Id

@GeneratedValue(strategy=GenerationType.SEQUENCE,generator="empid_generator")

**@SequenceGenerator(name="empid_generator",
sequenceName="employee_id",allocationSize=1)**

@Column(name="id",updatable=false,nullable=false)

private int id;

- sequenceName - Sequence created in DB.
- allocationSize - Sequence increment step value. Default is 50

hibernate.cfg.xml



- The hibernate.cfg.xml file should have **<mapping class>**.

```
<hibernate-configuration>
```

```
  <session-factory>
```

```
    <property name="hbm2ddl.auto">update</property>
```

```
      <property name="dialect">org.hibernate.dialect.Oracle9Dialect</property>
```

```
      <property name="connection.url">jdbc:oracle:thin:@localhost:1521:xe</property>
```

```
      <property name="connection.username">system</property>
```

```
      <property name="connection.password">tiger</property>
```

```
      <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
```

```
        <mapping class="com.lti.HiberAnnotations.Employee"/>
```

```
    </session-factory>
```

```
</hibernate-configuration>
```

Hibernate 5



- Hibernate 5 is the major release of Hibernate framework, which provides the support for JDK 8 and Special database.
- Hibernate provides the JPA(Java Persistence) support for NoSQL databases. It supports - MongoDB(Fongo)
- Bootstrap comes with release of Hibernate ORM.
- New bootstrap mechanism of the Hibernate SessionFactory has been introduced with many features.
- Although method of creating SessionFactory used in Hibernate 4 is still compatible.
- Java 8 Support
- Significant performance improvements

Hibernate 5 Session Factory



- SessionFactory plays a major role in Hibernate framework and takes the responsibility to provide session object to the client program in thread safe way.
- SessionFactory is heavy weight object and it is usually initialized at the start-up of the application.
- SessionFactory uses the information defined in the **hibernate.cfg.xml** file bootstrap Hibernate ORM environment.
- SessionFactory provides the Session object to Java program in thread safe way.

Package org.hibernate.boot



- This package contains the contracts that make up the Hibernate native bootstrapping API (building a SessionFactory).
- It is package for the process of bootstrapping a Hibernate SessionFactory.
- The first step in native bootstrapping is the building of a ServiceRegistry holding the services Hibernate will need during bootstrapping and at run time.

StandardServiceRegistry Class



- In Hibernate 5 StandardServiceRegistry class is used to create SessionFactory Object.

```
Configuration configuration = new Configuration();  
configuration.configure("hibernate.cfg.xml");  
SessionFactory factory = configuration.buildSessionFactory(new  
StandardServiceRegistryBuilder().configure().build());  
Session sess= factory.openSession();  
Transaction tx = sess.getTransaction();  
tx.begin();
```

Hibernate in Web Application



- Use JSP for presentation logic.
- Bean class for representing data and DAO class for database codes.
- Insert the record of the user in the database.

index.jsp

This page gets input from the user and sends it to the register.jsp file using post method.

```
<form action="register.jsp" method="post">  
Name:<input type="text" name="name"/><br><br>  
Password:<input type="password" name="password"/><br><br>  
  
Email ID:<input type="text" name="email"/><br><br>  
<input type="submit" value="register"/>  
</form>
```


Hibernate in Web Application- register.jsp



- This file gets all request parameters and stores this information into an object of User class. Further, it calls the register method of UserDao class passing the User class object.

```
<%@page import="com.csc.hibernate"%>
<jsp:useBean id="obj" class="com.csc.hibernate.User">
</jsp:useBean>
<jsp:setProperty property="*" name="obj"/>

<%
int i=UserDao.register(obj);
if(i>0)
out.print("You are successfully registered");

%>
```

Hibernate in Web Application- User.java



- It is the simple bean class representing the Persistent class in hibernate.

```
package com.csc.hibernate;  
  
public class User {  
    private int id;  
    private String name,password,email;  
  
    //getters and setters  
  
}
```

Hibernate in Web Application- user.hbm.xml



- It maps the User class with the table of the database.

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
<class name="com.csc.hibernate.User" table="emp100">
<id name="id">
<generator class="increment"></generator>
</id>
<property name="name"></property>
<property name="password"></property>
<property name="email"></property>
</class>

</hibernate-mapping>
```


Hibernate in Web Application- UserDao.java



- A Dao class, containing method to store the instance of User class.

```
package com.com.hibernate;

import org.hibernate.Session;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class UserDao {

    public static int register(User u){
        int i=0;
        Session session=new Configuration().
            configure().buildSessionFactory().open
            Session();
```

```
        Transaction t=session.beginTransaction();

        t.begin();

        i=(Integer)session.save(u);

        t.commit();
        session.close();

        return i; } }
```

Hibernate in Web Application- hibernate.cfg.xml



- It is a configuration file, containing informations about the database and mapping file.

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
<session-factory>
    <property name="dialect">org.hibernate.dialect.OracleDialect</property>
    <property name="connection.url">jdbc:oracle:thin:@localhost:1521:orcl</property>
    <property name="connection.username">system</property>
    <property name="connection.password">oracle</property>
    <property name="connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
    <mapping resource="user.hbm.xml"/>
</session-factory>
</hibernate-configuration>
```

THE END